

Rapport PIM

Prune Mamalet

Garance Werner-Guyader

December-Janvier 2024

Plan du rapport :

- 1 Objectif du projet
- 2 Architecture de l'application en modules
- 3 Présentation des choix réalisés
- 4 Présentation des algorithmes et types de données
- 5 Démarche de test
- 6 Difficultés rencontrés et solutions
- 7 Organisation de l'équipe
- 8 Bilan technique de l'avancement
- 9 Bilan personnel et individuel
- 10 Annexe

1 Objectif du projet

L'objectif de notre premier programme est de compresser un fichier texte à l'aide du codage d'Huffman. Il doit pouvoir afficher l'arbre et la table de codage d'Huffman qui permettent d'obtenir les codes correspondants aux symboles de notre texte.

Dans un second temps, notre deuxième programme doit pouvoir décompresser ce fichier compressé et renvoyer le texte initial.

2 Architecture de l'application en modules

2.1 Compresser

- module_compresser : spécification et corps et des fonctions nécessaire à la compression.
- test_module_compresser : module de tests des fonctions et procédures de module—compresser.
- lca : module qui gère la définition et modification des dictionnaires (LCA).
- compresser : programme qui compresses le fichier texte à l'aide des modules précédents.

2.2 Décompresser

- module_decompresser : spécification et corps et des fonctions nécessaire à la compression.
- decompresser : programme qui compresses le fichier texte à l'aide des modules précédents.
- test_decodage : module pour tester les fonctions de décodage, affichage et enregistrement du texte décodé à partir d'un arbre correctement reconstruit.

3 Présentation des choix réalisés

4 Présentation des algorithmes et types de données

4.1 Types de données

Compresser :

```
T_Arbre EST POINTEUR SUR T_Cellule2  
  
T_Cellule2 EST ENREGISTREMENT  
Symbole : Caractère  
Fréquence : Entier  
Code : T_Octet_Str  
Fils_Gauche : T_Arbre  
Fils_Droit : T_Arbre
```

Figure 1: type T_Arbre

Le Type Arbre permet de créer l'arbre d'Huffman en enregistrant le symbole et la fréquence et le code associés. Le type arbre est un pointeur vers un enregistrement (T_Cellule1) contenant les données souhaitées et les fils gauche et droit, eux même de type T_Arbre.

```
T_ListeChaine EST POINTEUR SUR T_Cellule  
  
T_Cellule EST ENREGISTREMENT  
Arbre : T_Arbre  
Suivant : T_ListeChaine
```

Figure 2: type T_ListeChaine

Le type ListeChaine permet d'enregistrer différents arbres provisoires et donc de former l'arbre d'Huffman par regroupement. C'est également un pointeur vers un enregistrement (T_Cellule2).

Décompresser :

```
T_Arbre EST POINTEUR SUR T_Cellule2  
T_Cellule2 EST ENREGISTREMENT  
  Symbole : Caractère  
  Code : T_Octet_Str  
  Fils_Gauche : T_Arbre  
  Fils_Droit : T_Arbre
```

Figure 3: type T_Arbre

Nous avons repris le même type de données que pour le compresser avec notre type Arbre. Il s'agit d'un pointeur vers un enregistrement ici seulement du Symbole, du code, du fils gauche et du fils droit, et plus de la fréquence qui ne nous sert pas.

4.2 Algorithmes

cf Annexe p10 pour les raffinages.

	Règle	Évaluation (I/P/A/+)	Justification/commentaire (optionnel)	Inacceptable (I)	Partiellement atteint (P)	Atteint (A)
Forme	Respect de la syntaxe Ri : Comment "... une action complexe ..." ? des actions combinées avec des structures de contrôle Rj : ...	A		Pas de numéro de raffinement (R0, R1, R2...) OU Pas de "Comment" OU Pas de rappel de l'action/expression complexe OU Pas de décomposition ou elle est mal indentée OU les Ri sont mentionnés dans l'algorithme complet (non séparés)	l'action/expression est donnée ET la décomposition est donnée MAIS Erreur sur le numéro (niveau) de raffinement OU Pas de décalage pour les actions	Le bon numéro pour le raffinement ET le "Comment" est là ET l'action/composé est donnée ET la décomposition est donnée indentée ET les actions sont décalées
	Verbe à l'infinitif pour les actions complexes	A		Pas ou peu de verbes à l'infinitif (moins de 20 %)	Moins de 70 % de verbes à l'infinitif	Tous les verbes sont à l'infinitif
	Nom ou équivalent pour expressions complexes	A		Utilisation de verbes à l'infinitif pour nommer une expression complexe	Moins de 10 % d'expressions complexes mal nommées	Aucun verbe à l'infinitif pour nommer une expression complexe
	Tous les Ri sont écrits contre la marge et espacés	A		Les Ri sont décalés en fonction de la valeur de i ET il n'y a pas de ligne blanche entre eux	Les Ri sont bien contre la marge OU 1 ou 2 lignes blanches séparent les Ri	Tous les Ri sont contre la marge ET 1 ou 2 lignes blanches les séparent
	Les flots de données sont définis	A		Pas de flot de données	Le flot de données est donné pour quelques actions ou expressions complexes OU les in et out ne sont pas donnés OU le type n'est pas donné lors de la première apparition d'une donnée	Les flots de données sont donnés pour presque toutes les actions ou expressions complexes ET Les in et les out sont présents ET le type d'une donnée est défini à sa première apparition
	Une seule structure de contrôle par raffinement	P		3 ou plus structures de contrôle (conditionnelle ou répétition) dans un raffinement	2 décisions ou répétitions au plus par raffinement	au plus 1 décision ou répétition par raffinement
	Pas trop d'actions dans un raffinement (moins de 6)	P		Beaucoup trop d'actions : > 9	Trop entre 6 et 9	Moins de 6
	Bonne présentation des structures de contrôle	A		Plus de 3 structures de contrôle sont mal présentées	1 ou 2 structures de contrôle sont mal présentées	Toutes les structures de contrôle sont présentées comme en cours

Fon d	Le vocabulaire est précis	P		On ne comprend pas plusieurs noms utilisés : pas significatifs OU utilisation d'abréviations OU mélange de plusieurs langues	On ne comprend pas quelques noms.	On comprend tous les noms ET pas d'abréviations ET pas de mélange de plusieurs langues
	Le raffinage d'une action décrit complètement cette action	P		Pour au moins 2 décompositions (raffinages) : Il manque des actions OU il manque des structures de contrôle (répétition, décision) OU il y a des actions interdites (comme Recommencer, Continuer en...)	Idem mais pour 1 décomposition seulement	Toutes les décompositions respectent les trois conditions.
	Le raffinage d'une action ne décrit que cette action	P		Pour au moins 2 décompositions : Il y a des actions en trop	Idem mais pour 1 décomposition seulement	Aucune action en trop dans aucune décomposition
	Les flots de données sont cohérents	A		Au moins 2 fois, on constate : des in avant des out pour des données locales OU des out sur des données en in de l'action complexe OU pas de out sur des données en out de l'action complexe OU des actions ne sont pas dans l'ordre vis à vis du flot	Idem mais on le constate 1 fois seulement	Les 4 règles sont toujours respectées
	Pas de structure de contrôle déguisée	A		Au moins 2 structures de contrôle déguisées (Faire ... si ..., Incrémenter tant que... calculer selon ...)	1 structure de contrôle déguisée	Aucune structure de contrôle déguisée
	Qualité des actions complexes	P		Au moins 2 fois, certaines actions ne sont pas explicites (i.e. on ne peut pas exécuter mentalement sans regarder le raffinage de certaines de ces actions) OU des actions pourraient être regroupées en une action complexe	Idem mais sur 1 raffinage seulement	Toujours respecté

5 Démarche de test

Pour chacune de nos fonctions de module_compresser, on a créé des fonctions de test à l'aide la commande pragma assert. En plus, on a demandé d'afficher régulièrement l'arbre pour vérifier cette fonction (Afficher_Arbre).

Lorsque tous les tests sont validés, le programme doit afficher "test ok".

Exemples :

```
procedure Tester_Suivant is
  Liste : T_ListeChaine;
begin
  Ajouter_Liste(Liste, Arbre1);
  Ajouter_Liste(Liste, Arbre2);
  pragma Assert (LArbre(Suivant(Liste))=Arbre2);
end Tester_Suivant;

procedure Tester_FinArbre is
begin
  pragma Assert(Fin_Arbre(Arbre1)=False);
  pragma Assert(Fin_Arbre(Arbre2)=True);
end Tester_FinArbre;
```

Figure 4: Tester suivant et fin de liste

```
procedure Tester_Fils is
  Fils_Droit : T_Arbre;
  Fils_Gauche : T_Arbre;
begin
  Initialiser_Arbre(Fils_Droit);
  Initialiser_Arbre(Fils_Gauche);
  Enregistrer_Arbre(Arbre2, p_1.To_Unbounded_String("1001010"), Fils_Gauche, Fils_Droit);
  Enregistrer_Arbre(Arbre2, 'a', 5, To_Unbounded_String("101101"), Fils_Gauche, Fils_Droit);
  Enregistrer_Arbre(Arbre2, p_2.To_Unbounded_String("1010"), Arbre0, Arbre2);
  pragma Assert(Arbre_Vide(Fils_Gauche_Arbre1));
  pragma Assert(Fils_Droit(Arbre1)=Arbre2);
  Recuperer_Fils_Gauche(Fils_Gauche, Arbre1);
  Recuperer_Fils_Droit(Fils_Droit, Arbre1);
  pragma Assert(Arbre_Vide(Fils_Gauche));
  pragma Assert(Fils_Droit = Arbre2);
  Afficher_Arbre(Arbre1);
end Tester_Fils;
```

Figure 5: Tester fils

Pour le module décompresser nous n'avons pas eu le temps de créer le module de test associé aux nouvelles fonctions que nous avons créés.

6 Difficultés rencontrée et solutions

Nous avons eu des difficultés pour trouver le bon type de données à utiliser pour pouvoir bien construire et représenter l'arbre d'Huffman. Nous avons finalement choisis un pointeur vers un enregistrement contenant toutes les données souhaitées (Symbole, Code, Fréquence, Fils gauche et Fils droit).

On a également eu du mal à comprendre ce qu'il fallait renvoyer dans le fichier compresser (des octets ou des codes en binaires ou des symboles etc) et comment faire la distinction entre les différentes parties (symboles utilisées, parcours infixé de l'arbre et texte codé) au moment de la décompression.

Enfin la fonction Afficher_Arbre qui affiche l'arbre d'Huffman a été difficile à conceptualiser et coder et nous a pris beaucoup de temps.

En commençant la décompression nous nous sommes rendues compte qu'il y avait un problème dans notre compression (pour la séparation entre le parcours et le texte codé) nous avons donc du reprendre compresser. Ensuite nous avons passé beaucoup de temps sur la reconstruction de l'arbre d'Huffman.

Nous n'avions pas réussi à recréer l'arbre d'Huffman à partir du parcours infixé au moment de l'oral, jeudi 16/01. Nous avons donc créé un module de test qui permet, si on a recréé l'arbre au préalable, de décoder le texte. En reprenant donc l'arbre d'Huffman du module compresser, on a pu vérifier que nos fonctions de décodage, affichage et enregistrement dans un nouveau fichier du texte décodé fonctionnaient.

Par la suite, on a finalement réussi à reprendre cette fonction de reconstruction de l'arbre (cf figure 7) et donc joindre les deux bouts de notre programme décompresser. Le fichier test_décodage n'est plus utile en tant que tel mais on a jugé important de le laisser puisque c'est une étape importante de notre avancement.

Désormais compresser.adb et décompresser.adb compilent et s'exécutent comme souhaité et fournissent bien un fichier test.txt.hff.d identique au fichier test.txt initial.

Nous utilisons le symbole \$ pour le fin de fichier et non \ \$ puisqu'il ne s'agissait pas d'un caractère et nous n'avons pas trouvé d'autres solutions pour résoudre ce problème. Il ne faut donc pas que notre fichier contienne le caractère \$.

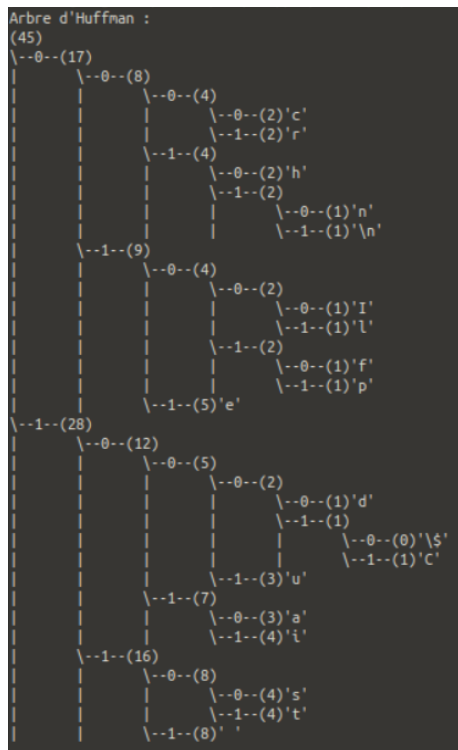


Figure 6: arbre d'Huffman construit par compresser.adb

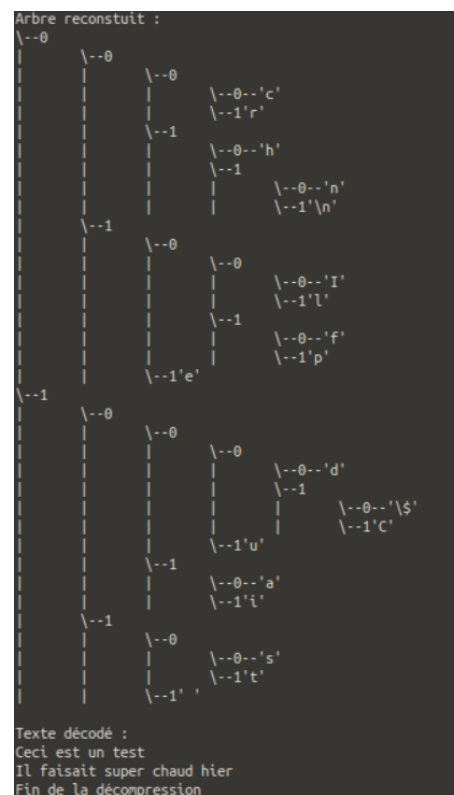


Figure 7: arbre d'Huffman reconstitue à partir du parcours infixe dans decompresser.adb

7 Organisation de l'équipe

	nom court	NOM	Prénom
Auteurs :	G	Werner--Guyader	Garance
	P	Mamalet	Prune

Modules	Sous-programmes	Codage	Test	Relecture
module_compresser	Initialiser	G P	G P	
	Longueur	P	/	
	Supprimer_Liste	G P	G P	
	Suivant	G P	G P	
	Fin_Liste	G P	G P	P
	LArbre	P	/	
	Recuperer_LArbre	P	/	
	Supprimer_Arbre	G P	G P	
	Supprimer_Cellule	P	-	
	Initialiser_Arbre	P	/	
	Enregistrer_Arbre	G P	G P	P
	Afficher_Arbre	G	G P	G
	Le_Fils_Gauche	G P	G P	
	Le_Fils_Droit	G P	G P	
	Recuperer_Fils_Gauche	P	P	
	Recuperer_Fils_Droit	P	P	
	Fin_Arbre	G P	G P	
	Arbre_Vide	P	P	
	Le_Code	P	/	
	Le_Symbole	P	/	
	La_Frequence	P	/	
	Modifier_Code	P	P	
	Modifier_Frequence	P	P	
	Ajouter_Liste	G P	G P	P
	Recuperer_Parcours_Arbre	G P	G P	P
	Afficher_Symbole	P	P	P
Compresser	Code	P	/	
	Creer_Dict	G P	/	
	Afficher_Couple	G P	/	
	Ecrire_Fichier	P	/	P
LCA	Concatener	P	-	P
	Cle_Actuelle	P	-	P
	Suivant	P	-	P
Nous avons réutilisés les fonctions que nous avons déjà écrites dans le projet 2				

Figure 8: Qui a fait quoi : compresser

Modules	Sous-programmes	Codage	Test	Relecture
Module_decompresser	Longueur_ListeSymbole	G		G
	Afficher_Caractere	G		
	Symbole_Correspondant	G		
	Les_Symboles	G		
	Modifier_Symbole	G		GP
	Reconstituer_arbre	P	GP	GP
	Decodage	G		
	Convertir_octet_bit	GP	GP	GP
	Changer_Symbole	G		
Nous avons réutilisés les fonctions que nous avons écrite dans compresser (sans la fréquence)				
Décompresser	Ecrire_Fichier_Decode	G		P
	Afficher_Texte_Decode	G		
Test_decodage	On a copié les deux sous programmes de Décompresser et utilisé l'arbre de compresser (G)			

Figure 9: Qui a fait quoi : décompresser

8 Bilan technique de l'avancement

Depuis le dernier rendu du rapport nous avons réussi à régler les problèmes de notre fonction afficher. Et réglé ce qui ne fonctionnait finalement pas dans compresser (le parcours de l'arbre d'Huffman notamment).

Nous n'avons cependant pas eu le temps de faire sorte que le programme soit robuste (notamment si le fichier que l'on cherche à compresser est vide ou n'existe pas). Nous ne pouvons pas compresser plusieurs fichier à la fois.

Pistes d'améliorations : mettre les fonctions qui sont définies dans la spécification du module compresser dans module_compresser.ads/.adb

Bilan du projet (résultats) :

```
test.txt (~Bureau/1SN/PIM/pr3/src) - VIM
Fichier Editer Affichage Rechercher Terminal Aide
1 Ceci est un test
2 Il faisait super chaud hier
```

Figure 10: fichier texte test.txt utilisé pour tester les programmes

```
prti5540erqggrnc:/Bureau/1SN/PIM/pr3/src$ xxd -b test.txt.hff
00000000: 00001011 01100011 01110010 01101000 01101110 00001010 .crhn.
00000006: 01001001 01101100 01100110 01110000 01100101 01100100 Ilfped
0000000c: 01000011 01110101 01100001 01101001 01110011 01110100 Cua1st
00000012: 00100000 00100000 00001101 01100011 01110000 10111011 .cp.
00000018: 00111100 01101100 01011111 10111100 11011111 00100110 <U/.&
0000001e: 11111010 11110011 01001110 10000100 11110101 01010101 .N...U
00000024: 11100101 01011110 11111100 10010101 10110001 11100000 .^...
0000002a: 01010101 00110000 11100101 01101100 01111100 01000000 U0.li@
```

Figure 11: fichier test.txt.hff créé par compresser.adb

```
test.txt.hff.d (~Bureau/1SN/PIM/pr3/src) - VIM
Fichier Editer Affichage Rechercher Terminal Aide
1 Ceci est un test
2 Il faisait super chaud hier
```

Figure 12: fichier décompressé test.txt.hff.d créé par décompresser

```
Bits_Parcours : 000011010110001101110000101110110011
```

Figure 13: parcours infixé de l'arbre renvoyé par compresser en mode bavard (-b)

9 Bilan personnel et individuel

Prune : J'ai du passer une après midi sur la conception des raffinages de *compresser* plus 3h pour les reprendre. J'ai passé 28 heures sur l'implémentation du code de *compresser* (dont 6 heures de projet en classe). Pour décompresser j'ai passé 15h30 sur le code de *décompresser*. Pour le rapport nous y avons pour l'instant passé 4 heures.

Ce projet m'a permis de voir comment s'organiser en binome sur un projet, et aussi comment gérer un projet assez conséquent. Il m'a aussi permis de comprendre comment gérer la lecture et l'écriture de fichier en Ada.

Garance : Peu de temps passé sur la conception des raffinages de *compresser* (je m'occupais de *décompresser*). Par la suite, environ 12h passé sur l'implémentation du code (6h de projet en classe et 6h pour le moment sur la fonction Afficher et d'autres corrections de code). J'ai passé une après midi sur les raffinages de *décompresser* et 24h sur le code. Quant au rapport, on a mis 4h pour l'instant.

Ce projet m'a permis de mieux appréhender et comprendre en profondeur le langage. J'ai aussi pu apprendre à travailler sur du long terme en binome et gérer mon temps.

10 Annexes

10.1 Raffinages

Compresser :

T_ListeChaine EST POINTEUR SUR T_Cellule

T_Cellule EST ENREGISTREMENT

Arbre : T_Arbre

Suivant : T_ListeChaine

T_Arbre EST POINTEUR SUR T_Cellule2

T_Cellule2 EST ENREGISTREMENT

Symbole : Caractère

Fréquence : Entier

Code : T_Octet_Str

Fils_Gauche : T_Arbre

Fils_Droit : T_Arbre

T_ListeChaine EST POINTEUR SUR T_Cellule

T_Cellule EST ENREGISTREMENT

Arbre : T_Arbre

Suivant : T_ListeChaine

T_Arbre EST POINTEUR SUR T_Cellule2

T_Cellule2 EST ENREGISTREMENT

Symbole : Caractère

Code : T_Octet_Str

Fils_Gauche : T_Arbre

Fils_Droit : T_Arbre

R0 : Compresser un fichier texte d'après le codage Huffman

R1 : Comment "Compresser un fichier texte d'après le codage Huffman" ?

Récupérer le fichier d'origine

Fichier : **out File**

Déterminer les fréquences d'apparition des caractères

Fichier : **in File**;

Liste_Frequence: **out T_ListeChaine**

Construire l'arbre de Huffman

Liste_Frequence: **in**

ArbreHuffman : **out T_Arbre**

Bit_Parcours_infixe : **out Str**

Créer la table de codage

ArbreHuffman : **in**

	Dict_Codage : out LCA Bits_Parcours_infixe : out Str
Coder le texte	Fichier : in File , Texte_Code : out Str
Ecrire le fichier compressé	Dict_Codage, Bits_Parcours_infixe, Texte_Code : in Fichier_Code : out File

R2 : Comment "Récupérer le fichier d'origine" ?

Récupérer nom du fichier
Récupérer argument d'affichage
Ouvrir le fichier texte

R2 : Comment "Déterminer les fréquences d'apparition des caractères" ?

Initialiser le tableau de fréquence	Liste_Frequence : out T_ListeChaine
TantQue not Fin_Fichier Alors	
Récupérer le caractère	Caractère : out caractère
Mettre à jour la fréquence	Liste_Frequence : in out
FinTQ	

R2 : Comment "Construire l'arbre de Huffman" ?

TantQue Longueur(Tableau_freq) > 1 Faire	
Rechercher 2 plus faibles fréquences	Liste_Frequence: in T_ListeChaine L_Arbre1, L_Arbre2 : out T_Arbre
Construire la nouvelle Arbre	NouvelArbre : out T_Arbre
Mettre à jour l'arbre de Huffman	Liste_Frequence : in out
FinTQ	
Récupérer l'Arbre	ArbreHuffman : out T_Arbre

R2: Comment : "Créer la table de codage" ?

Initialiser Bits_Parcours_Infixe	
Coder	Arbre : in out T_Arbre , Code_v : in Str , Bits_Parcours_Infixe : in Str
Créer dictionnaire	Arbre : in T_Arbre , Dict_Codage : in out T_LCA

R2 : Comment : "Coder le texte" ? Fichier : **in File** , Texte_Code : **out Str**

Initialiser Texte_Code

TantQue not Fin_Fichier **Alors**

 Récupère caractère

 Récupère code huffman associé à caractère Symbole : **in Character** , Dict_codage: **in out**

 Ajouter code à Texte_Code

Fin TantQue

R2 : Comment "Ecrire le fichier compressé" ?

 Ajouter liste de caractères

 Ajouter Parcours infixe et texte codé

R3 : Comment "Mettre à jour la fréquence" ?

 Curseur <- Tableau_freq

Répéter

 Passer à l'élément suivant

Jusqu'à Curseur.all.Arbre.Symbole = Caractère or Curseur = Null

Si Curseur = Null **Faire**

 Créer nouvel arbre

 Nouvel_Arbre : **out T_Arbre**

 Créer Nouvelle Cellule

 Nouvel_Arbre : **in T_Arbre**

 Nouvelle_Cellule : **out T_Cellule**

Sinon

 Curseur.all.Arbre.Frequence <- Curseur.all.Arbre.Frequence + 1

FinSi

R3 : Comment "Rechercher 2 plus faibles fréquences" ?

Initialiser les variables de recherche des minimums

TantQue Curseur $\neq$ null **Faire**

Arbre_actuel <- Curseur.all.Arbre

Si Arbre_actuel.Frequence < Min2 **Faire**

Min2 <- Arbre_actuel.Frequence

Cellule2 <- Arbre_actuel

FinSi

Si Min1 > Min2 **Faire**

Inverser les 2 minimums

FinSi

Curseur <- Curseur.all.Suivant

Fin Tant Que

R3 : Comment "Construire la nouvel Arbre" ? => Fonction correspond à la fonction Enregistrer_Arbre

Nouvelle_Branche : T_Branche

Nouvelle_Branche.all.Symbole <- '0'

Nouvelle_Branche.all.Frequence <- Min1 + Min2

Nouve_Branche.all.Code <- 0

Nouvelle_Branche.all.Fils_Gauche <- Branche1.all.Arbre

Nouvelle_Branche.all.Fils_Droit <- Branche2.all.Arbre

R3 : Comment "Mettre à jour l'arbre de Huffman" ?

Ajouter la nouvelle branche

Supprimer Cellule1 et Cellule2

R3 : Comment "Récupérer l'arbre" ?

```
Arbre <- Liste_Frequence.all.Arbre
```

R3 : Comment "Coder" ?

Si Fin_Arbre (Arbre) **Alors**

```
Arbre.all.Code <- Code_v
```

Sinon

```
FG <- Arbre.all.Fils_Gauche
```

```
FD <- Arbre.all.Fils_Droit
```

```
Coder      FG: in out T_Arbre, Code_v&"0" : in Str, Bits_Parcours_Infixe&"0" : in Str
```

```
Coder      FD: in out T_Arbre, Code_v&"1" : in Str, Bits_Parcours_Infixe&"1" : in Str
```

FinSi

R3 : Comment "Créer dictionnaire" ?

Arbre : **in T_Arbre**, Dict_Codage : **in out T_LCA**

Si Fin(Arbre) **Alors**

```
Enregistrer Symbole Code dans le dictionnaire
```

Sinon

```
Créer dictionnaire      Arbre.all.Fils_Gauche in T_Arbre, DictFG in out T_LCA
```

```
Créer dictionnaire      Arbre.all.Fils_Droit in T_Arbre, DictFD in out T_LCA
```

```
Rassembler FG et FD
```

FinSi

R3 : Comment "Ajouter la liste de caractères"

```
Enregistrer la position du caractère de fin
```

```
Enregistrer les autres caractères
```

R3 : Comment "Ajouter Parcours infixe et texte codé" ?

Ajouter les zéros nécessaires

Enregistrer octet par octet

R4 : Comment "Initialiser les variables de recherche des minimums" ?

Si Liste_Frequence.all.Arbre.Frequence < Liste_Frequence.all.Suivant.Arbre.Frequence **Faire**

```
L_Arbre1 <- Liste_Frequence
Min1 <- L_Arbre1.all.Fréquence
L_Arbre2 <- Liste_Frequence.all.Suivant
Min2 <- L_Arbre2.all.Fréquence
Curseur <- L_Arbre2.all.Suivant
```

Sinon

```
L_Arbre2 <- Liste_Frequence
Min2 <- L_Arbre2.all.Fréquence
L_Arbre1 <- Liste_Frequence.all.Suivant
Min1 <- L_Arbre1.all.Fréquence
Curseur <- L_Arbre1.all.Suivant
```

FinSi

R4 : Comment "Créer un nouvel arbre" ? => procédure enregistrer_arbre

```
Nouvel_Arbre.all.Symbole <- Caractère
Nouvel_Arbre.all.Frequence <- 1
Nouvel_Arbre.all.Fils_Gauche <- Null
Nouvel_Arbre.all.Fils_Droit <- Null
```

R4 : Comment "Créer nouvelle Cellule" ?

```
Nouvelle_Cellule.all.Suivant <- Null
Nouvelle_Cellule.all.Arbre <- Nouvel_Arbre
Curseur.all.Suivant <- Nouvelle_Cellule
```


R4 : Comment "Supprimer Cellule" ?

Liste : **in out T_ListeChaine**, Element_a_supprimer : **in T_ListeChaine**

Si Liste = Element_a_supprimer **Alors**

Liste <- Liste.all.Suivant

Sinon

Parcours <- Liste

TantQue Parcours.all.Suivant ≠ Element_a_supprimer **Faire**

Parcours <- Parcours.all.Suivant

FinTQ

Parcours.all.Suivant := Element_a_supprimer.all.Suivant

FinSi

R4 : Comment "Ajouter nouvelle branche" ?

Nouvelle_Cellule.Suivant <- Liste_Feuille

Liste_Feuille <- Nouvelle_Cellule

R4 : Comment "Enregistrer la position du caractère de fin" ?

Position <- 0

Avancee_Dict <- Dict_Codage

TantQue not Est_Vide(Avancee_Dict) **Faire**

Cle <- La_Cle(Avancee_Dict)

Si Cle ≠ '\$' **Alors**

Position <- Position + 1

Sinon

Ecrire position dans le fichier

FinSi

Avancee_Dict <- Suivant(Avancee_dict)

Fin TQ

R4 : Comment "Enregistrer les autres caractères" ?

Avancee_Dict <- Dict_Codage

TantQue not Est_Vide(Avancee_Dict) **Faire**

Cle <- La_Cle(Avancee_Dict)

Si Cle ≠ '/' **Alors**

Ecrire octet du caractère

FinSi

Avancee_Dict <- Suivant(Avancee_dict)

FinTQ

R4 : Comment "Ajouter les zéros nécessaires" ?

A_Enregistrer <- Bits_Parcours_Infixe & Texte_Code

Nb_bits_Enregistrer <- Longueur(A_Enregistrer)

Nb_zeros_aajouter <- 8 - Nb_bits_Enregistrer mod 8

Pour k **De** 1 **à** Nb_zeros_aajouter **Faire**

A_Enregistrer <- A_Enregistrer & "0"

FinPour

Nb_bits_Enregistrer <- Longueur(A_Enregistrer)

R4 : Comment "Enregistrer octet par octet" ?

Indice_Curseur <- 1

Tant Que Indice_Curseur <= Nb_bits_Enregistrer **Faire**

Ecrire l'octet correspondant à la chaîne A_Enregistrer entre les indices Indice_Curseur et Indice_Curseur+7

Indice_Curseur <- Indice_Curseur + 8

Fin TQ

Décompresser

T_Arbre EST POINTEUR SUR T_Cellule2

T_Cellule2 EST ENREGISTREMENT

Symbole : Caractère

Code : T_Octet_Str

Fils_Gauche : T_Arbre

Fils_Droit : T_Arbre

R0 : Décompresser un fichier compressé à l'aide de l'algo d'Huffman

R1 : Comment « Décompresser un fichier compressé à l'aide de l'algo d'Huffman » ?

Lire fichier compresseur

Nom_Fichier : **in Unbounded_String**

Liste_symbole , Parcours_texte : **out Unbounded_String**

Reconstruire le squelette de l'arbre d'Huffmann

Arbre_Huffman : **out T_Arbre**

Parcours_texte : **in Unbounded_String**

Mettre le symbole dans chaque cellule de l'arbre

Arbre_Huffman : **in out T_Arbre**

Symbole_utilisees : **in Unbounded_String**

Décoder le fichier et enregistrer dans le fichier .d

Parcours_texte : **in Unbounded_String**

Texte_Decode : **out Unbounded_String**

Arbre_Huffman : **in T_Arbre**

R2 : Comment « Reconstruire le squelette de l'arbre d'Huffman » ?

Initialiser un Arbre Nul et Arbre_Huffman

Enregistrer la première cellule de Arbre_Huffman

Reconstruire Arbre_Huffman à partir du parcours infixe de l'arbre

R2: Comment “Mettre le symbole dans chaque cellule de l’arbre ” ?

Curseur <- Arbre_Huffman

Si Curseur /= null **Alors**

FG <- Curseur.all.Fils_Gauche

FD <-Curseur.all.Fils_Droit

Remplacer les symboles de [l'Arbre] en parcourant la liste des symboles utilisés

Fin Si

R2 : Comment « Décoder le fichier et enregistrer dans le .d » ?

texte_decode : Unbounded_String

texte_code <- parcours_texte

i : Integer <- 0 ;

Tant que Symbole /= '\$' **Faire**

Tant Que texte_code(i) /= « . » **Faire**

Ecrire le symbole dans le fichier

i := i +1;

FinTQ

i<-i+1

Liste_Symbole := Liste_Symbole & Symbole

FinTQ

Enregistrer Liste_Symbole dans *exemple-sujet.txt.hff.d*.

Afficher le texte décodé

R3 : Comment “Enregistrer un Arbre_Huffman contenant une seule cellule” ?

Enregistrer_Arbre *déjà raffiné dans compresser*

R3 : Comment “Reconstruire Arbre_Huffman à partir du parcours infixe de l’arbre”?

Initialiser un Arbre Nul

Si Fin = False **Alors**

Lire le bit actuel

Enregistrer les Fils correspondants

Fin Si

R3: Comment “Remplacer les symboles de [l'Arbre] en parcourant la liste des symboles utilisés” ?

Si Fin_Arbre(Curseur) **Alors**

Modifier le symbole en fonction de l'indice

Sinon

Remplacer les symboles de [FG] en parcourant la liste des symboles utilisés

Remplacer les symboles de [FD] en parcourant la liste des symboles utilisés

FinSi

R3 : Comment « Enregistrer Liste_Symbole dans *exemple-sujet.txt.hff.d.* » ?

Nom_Fichier_DeCode <- Nom_Fichier & « .d »

Create(File, Out_File, Nom_Fichier_DeCode);

S <- Stream(File) ;

Ajouter un caractère dans le fichier .hff.d

Close(File) ;

R3 :Comment « Afficher le texte décodé » ?

Si « -s » Alors

Null ;

Sinon

Pour i de 1 à Longueur(texte_decode) Faire

Put(Symbole(i),1) ;

Fin Pour

Fin Si

R4 : Comment : Enregistrer les Fils correspondants" ?

Si Bit_actuel = '0' Alors

Gérer Bit_Actuel à '0'

Sinon

Gérer Bit_actuel à 1

FinSi

R4 : Comment "Modifier le symbole en fonction de l'indice" ?

SI indice = Position_Dollard Alors

Curseur.all.Symbole <- '\$'

Sinon

Curseur.all.Symbole <- Symbole_utilises(indice)

FinSi

indice <- indice + 1

Sinon

Enregistrer Sans_FD^Fils_Droit

Reconstruire [Sans_FD^Fils_Droit] à partir du parcours infixe de l'arbre

FinSi

R5 : Comment "Regarder le bit suivant et reconstruire en fonction" ?

Si Bit_Suivant = '0' **Alors**

Reconstruire Fils_Gauche de Arbre_Actuel à partir de la suite du parcours infixe de l'arbre

Sinon

Reconstruire Arbre_Actuel à partir de la suite du parcours infixe de l'arbre

Fin Si

R5 : Comment "Regarder si on est à la racine ou non avant d'enregistrer/reconstruire"

Si Arbre_Actuel = Arbre_Huffman et Arbre_Actuel.all.Fils_Droit = null **Alors**

Fin <- True

Sinon

Trouver le noeud sans Fils Droit (Sans_Fd : in T_Arbre)

Enregistrer L'arbre vide de racine le noeud sans fils droit (Enregistrer_Arbre)

Reconstruire Fils_Droit de Sans_FD à partir de la suite du parcours infini de bits

Fin Si

R6 : Comment : " Trouver sans fils droit " ?

Curseur <- Arbre

Si Curseur /= null **Alors**

Si Curseur^Fils_Gauche /= null **Alors**

Trouver sans fils droit [Curseur^fils_gauche]

Si Trouve /= false **Alors**

Si Curseur^Fils_Droit = null **Alors**

Sans_FD <- Curseur

Trouve <- True

Sinon

Trouver sans fils droit [Curseur^Fil_droit]

FinSi

FinSi

FinSi